

Conjugate Gradient Algorithm (CG)

Inputs: $A \in \mathbb{R}^{n \times n}$ SPD, $v \in \mathbb{R}^n$, $K = \# \text{iterations}$

$$x_0 = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$r_0 = v$$

$$d_0 = r_0$$

for $j = 1, 2, \dots, K$

$$\alpha_j = r_{j-1}^T r_{j-1} / (d_{j-1}^T A d_{j-1})$$

$$x_j = x_{j-1} + \alpha_j d_{j-1}$$

$$r_j = v - A x_j = r_{j-1} - \alpha_j A d_{j-1}$$

$$\beta_j = (r_j^T r_j) / (r_{j-1}^T r_{j-1})$$

$$d_j = r_j + \beta_j d_{j-1}$$

endfor

$$d_0, r_0 \in K_1(A, v)$$

$$x_1 \in K_1(A, v)$$

By induction: $d_{j-1}, r_{j-1}, x_j \in K_j(A, v)$

Proof: $j = 1$: $d_0, r_0, x_1 \in K_1(A, v) = \text{span}(v)$

Assume that $d_{j-1}, r_{j-1}, x_j \in K_j(A, v)$

$$\Rightarrow r_j = \underbrace{r_{j-1}}_{\in K_j} - \alpha_j \underbrace{A d_{j-1}}_{\substack{\in K_j \\ \in K_{j+1}}} \in K_{j+1}(A, v)$$

$$d_j = \underbrace{r_j}_{\in K_{j+1}} + \beta_j \underbrace{d_{j-1}}_{\in K_j} \in K_{j+1}(A, v)$$

$$x_{j+1} = \underbrace{x_j}_{\in K_j} + \alpha_{j+1} \underbrace{d_j}_{\in K_{j+1}} \in K_{j+1}(A, v). \quad \square$$

total cost: $O(nK + K \cdot \text{matvec})$

OPTIMALITY PROPERTY OF CG:

x_j is the best approximation to the solution $x_* = A^{-1}v$, in the A -norm, among all vectors in $K_j(A, v)$.

$$x_j = \underset{x \in K_j(A, v)}{\text{argmin}} \|x - x_*\|_A$$

Def: Given a matrix $A \in \mathbb{R}^{n \times n}$, and $v \in \mathbb{R}^n$, the j -th KRYLOV SUBSPACE is

$$K_j(A, v) = \text{span}\{v, Av, A^2v, \dots, A^{j-1}v\}$$

$$= \{w \in \mathbb{R}^n \text{ s.t. } w = (c_0 I + c_1 A + c_2 A^2 + \dots + c_{j-1} A^{j-1})v \text{ for some coefficients } c_0, c_1, \dots, c_{j-1} \in \mathbb{R}\}$$

$$\rightsquigarrow K_1(A, v) = K_2(A, v) = \text{span}(v, Av) = \text{span}(v, \lambda v) = \text{span}(v)$$

$$K_1(A, v) \subseteq K_2(A, v) \subseteq K_3(A, v) \subseteq \dots$$

$$\dim(K_j(A, v)) \leq j$$

Q: Can you find A, v s.t. $K_1(A, v) = K_2(A, v)$?

$$A = I, v \in \mathbb{R}^n \setminus \{0\}$$

$$K_1(I, v) = \text{span}(v)$$

$$K_2(I, v) = \text{span}(v, I \cdot v) = \text{span}(v)$$

$$\text{If } Av = \lambda v \quad (\lambda, v) \text{ eigenpair of } A$$

Gradient Descent (GD) algorithm

$r_0 = v$
 for $j=1, \dots, K$
 $\alpha_j = r_{j-1}^T r_{j-1} / (r_{j-1}^T A r_{j-1})$
 $x_j = x_{j-1} + \alpha_j r_{j-1}$
 $r_j = r_{j-1} - \alpha_j A r_{j-1}$
 end

Ex: $r_{j-1}, x_j \in K_j(A, v)$
 [Show it by induction!]

$\leadsto$ consequence for this:

$$\|x_j^{CG} - x_*\|_A \leq \|x_j^{GD} - x_*\|_A \quad \text{for all } j$$

recall: $\|y\|_A = \sqrt{y^T A y}$

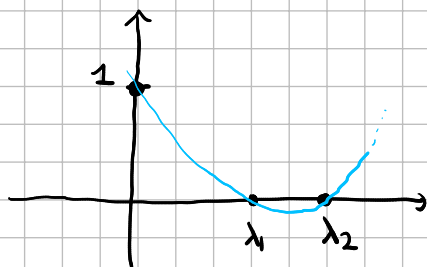
vectors z, y are A -orthogonal if $y^T A z = 0$

Theorem: If CG does not break down, then

- $d_0, d_1, d_2, \dots$ are A -orthogonal ($d_i^T A d_j = 0$ for $i \neq j$)
- $r_0, r_1, r_2, \dots$ are orthogonal ($r_i^T r_j = 0$ for $i \neq j$)

Theorem: $\|x_j - x_*\|_A \leq \|x_0 - x_*\|_A \cdot \min_{\substack{q(t) \text{ polynomial} \\ \text{of degree } \leq j \\ \text{such that } q(0)=1}} \max_{\substack{\lambda_1, \dots, \lambda_n \\ \text{eigenvalues} \\ \text{of } A}} |q(\lambda_i)|$

Example: if A has 2 distinct eigenvalues:



$\leadsto \exists q(t)$ of degree 2 s.t.
 $q(0)=1, q(\lambda_1)=q(\lambda_2)=0$

$\leadsto$ theorem tells us that
 CG converges to
 the true solution in 2 iterations!

Ex: Given A , ^{SPD} can you find c_1, c_2 s.t. $c_1 \|x\| \leq \|x\|_A \leq c_2 \|x\|$?

$$\lambda_{\min}(A) \underbrace{x^T x}_{\|x\|^2} \leq \underbrace{x^T A x}_{\|x\|_A^2} \leq \lambda_{\max}(A) \cdot \underbrace{x^T x}_{\|x\|^2} \quad (\text{for any } A \in \mathbb{R}^{n \times n} \text{ symmetric})$$

$$\Rightarrow \sqrt{\lambda_{\min}(A)} \|x\| \leq \|x\|_A \leq \sqrt{\lambda_{\max}(A)} \|x\|$$

Proof of the theorem: ($x_0 = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$).

$x_j \in K_j(A, v) \Leftrightarrow x_j = p(A) \cdot v$ for some polynomial p of degree $\leq j-1$.

$$\begin{aligned} \|x_j - x_*\|_A &= \|p(A)v - x_*\|_A = \|p(A)Ax_* - x_*\|_A = \|(p(A)A - I) \cdot x_*\|_A \\ &= \|-q(A)x_*\|_A, \quad \text{where } q(t) = -p(t)t + 1 \leadsto q(t) \text{ degree } \leq j, \quad q(0)=1 \end{aligned}$$

$\hookrightarrow$ we wrote the error at step j as $\|-q(A)x_*\|_A$

$q(t)$ is the polynomial of degree $\leq j$, with $q(0)=1$ that minimizes $\|-q(A)x_*\|_A$

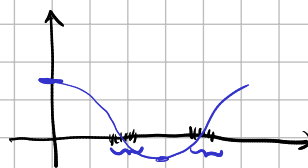
Take a spectral decomposition $A = U D U^T$ ↗ orthogonal
 $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$

Define $y_* = U^T x_*$ ($x_* = U y_*$)

$$\begin{aligned} \|q(A)x_*\|_A &= \|U \cdot q(D) U^T U y_*\|_A = \|U \cdot \begin{bmatrix} q(\lambda_1) & & \\ & q(\lambda_2) & \\ & & \ddots & \\ & & & q(\lambda_n) \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}\|_A \\ &= \sqrt{[y_1 \ y_2 \ \dots \ y_n] \begin{bmatrix} q(\lambda_1) & & \\ & \ddots & \\ & & q(\lambda_n) \end{bmatrix} \underbrace{U^T A U}_D \begin{bmatrix} q(\lambda_1) & & \\ & \ddots & \\ & & q(\lambda_n) \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}} \\ &= \sqrt{[y_1 \ y_2 \ \dots \ y_n] \begin{bmatrix} \lambda_1 q(\lambda_1)^2 & & \\ & \ddots & \\ & & \lambda_n q(\lambda_n)^2 \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}} \\ &= \sqrt{\lambda_1 q(\lambda_1)^2 y_1^2 + \dots + \lambda_n q(\lambda_n)^2 y_n^2} \leq \max_{t \in \{\lambda_1, \dots, \lambda_n\}} |q(t)| \cdot \sqrt{\lambda_1 y_1^2 + \dots + \lambda_n y_n^2} \\ &= \|x_*\|_A \end{aligned}$$

let's see why: $\|x_*\|_A = \|U y_*\|_A = \sqrt{y_*^T U^T A U y_*}$
 $= \sqrt{y_*^T D y_*} = \sqrt{\lambda_1 y_1^2 + \dots + \lambda_n y_n^2} \quad \square$

Example: clustered eigenvalues help!



Theorem: For CG, $\|x_j - x_*\|_A \leq 2 \|x_0 - x_*\|_A \cdot \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^j$
 where $\kappa = \text{condition number of } A = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$

Big problem of CG: for some matrices (especially ones with large $\kappa(A)$) the convergence is REALLYYY slow!

→ a powerful idea to address this is PRECONDITIONING

$$Ax = b \quad \rightsquigarrow \quad \underbrace{PA}x = Pb \quad \text{for an invertible matrix } P$$

⇔

$$PAP^T (P^{-T}x) = Pb$$

↳ Solve linear system $\underbrace{B}_{B=PAP^T} y = \underbrace{\tilde{b}}_{\tilde{b}=Pb}$ and then recover $x = P^T y$

If we are able to find a matrix P such that PAP^T is "good for CG" (also clustered eigenvalues) (well conditioned, small $\kappa(PAP^T)$)

then we can solve $(PAP^T)y = Pb$ with CG and

then compute $x = \underbrace{P^T}_{\text{transpose}} y$

$\leadsto$ the ideal preconditioner is cheap to apply (and also its transpose) to vectors, and is such that PAP^T has clustered eigenvalues.

Example : $P = I \leadsto$ nothing happens!

$A = LL^T$ Cholesky factorization (L is lower triangular)

$\leadsto P = L^{-1}$

$$PAP^T(P^{-T}x) = Pb$$

$$\begin{aligned} \hookrightarrow L^{-1}(LL^T)L^{-T}L^Tx &= L^{-1}b \\ L^Tx &= L^{-1}b \end{aligned}$$

$\hookrightarrow$ it would be good, but once we compute a Cholesky decomposition, we would directly solve the original linear system!

Incomplete Cholesky factorization $\leadsto$ a way to choose a preconditioner (the idea is that computing Cholesky for a sparse A would be too expensive and would result in a lot of fill-in $\Rightarrow$ we throw away some of the entries of the L factor).